SECURITY ARCHITECTURE & ENGINEERING

Cyber Security Reference Guide

Top Critical Application Vulnerabilities (2025 Standard)

Document Classification: Internal Technical Reference • **Target Audience:** Developers, DevOps, & Security Auditors

Modern application security requires continuous vigilance across all stages of the software development lifecycle (SDLC). This reference guide compiles and breaks down the top 10 most critical security vulnerabilities defined for 2025. By understanding the root causes, operational impacts, and robust remediation strategies for each vulnerability, engineering teams can build resilient systems that protect organizational data and maintain operational integrity.

Critical Vulnerability Directory

A01:2025 – Broken Access Control

Description: Occurs when an application fails to properly restrict what authenticated users are allowed to do. Attackers exploit these flaws to access unauthorized pages, view or modify other users' sensitive data, elevate privileges, or perform administrative actions without permission.

Remediation:

- Implement a centralized, server-side authorization mechanism based on roles or attributes (RBAC/ABAC).
- Deny access by default (Fail-Secure architecture).
- Ensure all identifiers used in APIs (IDs, UUIDs) are validated against user ownership permissions on every request.

Exploitation Scenario:

A normal user alters the account parameter directly in the URL query string (e.g., changing `/account?id=1002` to `/account?id=1001`) to view another user's private dashboard without validation.

A02:2025 – Security Misconfiguration

Description: Happens when applications, servers, cloud infrastructure, or databases are configured insecurely. Common gaps include keeping default factory credentials, leaving unnecessary features/ports enabled, misconfiguring permissions, displaying descriptive error logs to users, and missing foundational HTTP security headers.

Remediation:

- Implement automated hardening scripts and continuous configuration compliance tracking.
- Disable unneeded services, endpoints, and default accounts across the ecosystem.
- Enforce secure HTTP headers (e.g., CSP, HSTS, X-Content-Type-Options) globally.

Exploitation Scenario:

An enterprise deployment leaves an open administration console accessible via the public internet with default factory credentials (e.g., `admin / admin`), allowing instant external control.

A03:2025 – Software Supply Chain Failures

Description: Arises when applications depend on compromised, malicious, or unvetted third-party software components, packages, open-source libraries, or build pipeline tools. Attackers inject vulnerabilities or malicious packages early in the stream to compromise down-river applications automatically.

Remediation:

- Generate and continuously monitor Software Bills of Materials (SBOM) for all production systems.
- Utilize automated Software Composition Analysis (SCA) tools to flag vulnerable dependencies during build pipelines.
- Pin specific versions and cryptographically sign internal artifacts to protect build pipelines.

Exploitation Scenario:

An attacker gains control over a popular open-source utility package on npm, publishing a patched minor update that covertly extracts and transmits cloud database credentials to a remote server.

A04:2025 – Cryptographic Failures

Description: Occurs when highly sensitive data (passwords, health records, credit cards) is compromised due to weak encryption algorithms, poor key management, obsolete cryptographic protocols, or transmitting private data over cleartext connections.

Remediation:

- Classify data sensitivity and encrypt all data at rest and in transit using modern, industry-standard protocols (e.g., TLS 1.3, AES-256).
- Store passwords using strong, adaptive salted hashing algorithms such as Argon2id or bcrypt.
- Implement robust key rotation policies and secure key management systems (KMS).

Exploitation Scenario:

An application stores core user passwords using an outdated, fast algorithm like MD5 or as plain text within an unencrypted backup file, allowing immediate exposure during an incident.

A05:2025 – Injection

Description: Vulnerabilities occur when untrusted user input is directly concatenated into queries or commands, forcing the interpreter to execute unintended instructions. This category includes SQL, NoSQL, OS Command, and LDAP injections.

Remediation:

- Enforce strict separation of data from commands using parameterized queries and prepared statements.
- Apply strict context-aware output encoding and allowlist-based input validation.
- Execute application queries using the principle of least privilege within database engines.

Exploitation Scenario:

An attacker bypasses standard application login portals entirely by typing a crafted string like `' OR '1'='1` into a login field, altering the underlying SQL syntax to return a valid user record.

A06:2025 – Insecure Design

Description: Refers to structural or architectural flaws introduced before code is even written. It represents a fundamental absence of threat modeling and robust security design patterns, which cannot be fixed by implementation updates alone.

Remediation:

- Integrate secure design practices and systematic threat modeling early into the initial feature planning phases.
- Enforce strict boundary separation and implement multi-layer validation controls for high-risk corporate business flows.
- Establish and use secure, pre-approved architectural component patterns throughout development teams.

Exploitation Scenario:

An online financial platform is designed without a mandatory dual-authorization or transaction approval mechanism, enabling a compromised single account to drain substantial assets without secondary verification.

A07:2025 – Authentication Failures

Description: Occurs when session validation or user identity checks are poorly constructed. This allows automated script tools or attackers to compromise active login sessions, run automated credential stuffing operations, or easily hijack accounts.

Remediation:

- Mandate multi-factor authentication (MFA) across all user accounts, especially administrative assets.
- Enforce strict rate-limiting and account lockouts to prevent credential stuffing and brute-force attacks.
- Deploy cryptographically secure, randomized session IDs with explicit, short server-side lifespans.

Exploitation Scenario:

An internet-facing API lacks login rate-limiting controls, allowing an attacker to execute automated brute-force scripts using millions of common password combinations without triggering an account lockout.

A08:2025 – Software or Data Integrity Failures

Description: Happens when critical updates, dynamic plugins, parameter configurations, or serialized data blocks are read by applications without verification checks. Attackers alter these streams to run unauthorized code or tamper with critical logic.

Remediation:

- Utilize verified digital signatures or cryptographic hash checks (e.g., SHA-256) across all external updates and downloads.
- Avoid passing serialized objects across untrusted boundaries; instead, use standardized, raw data formats like JSON.
- Incorporate rigorous verification pipelines to confirm data hasn't been altered mid-flight.

Exploitation Scenario:

A desktop application automatically pulls and runs server patch binaries over unencrypted channels without checking digital signatures, enabling a malicious actor to inject a customized payload.

A09:2025 – Logging & Alerting Failures

Description: Occurs when important security-relevant events—such as login rejections, authorization steps, or server-side exceptions—are not captured or actively monitored. This prevents detection of ongoing intrusions and severely impairs forensic incident response.

Remediation:

- Ensure all authentication, authorization, and severe errors are logged with complete user and contextual metadata.
- Stream application logs securely to a centralized Security Information and Event Management (SIEM) system.
- Configure threshold-based operational alerts to trigger rapid remediation during abnormal traffic patterns.

Exploitation Scenario:

An application undergoes a massive automated credential-stuffing attack for several weeks, but because bad login attempts are completely omitted from logs, administrators remain unaware until data is leaked.

A10:2025 – Mishandling of Exceptional Conditions

Description: Occurs when applications fail to safely catch internal system errors or operational failures. Unhandled exceptions often leak structural architecture maps, reveal database versions, or cause unexpected state changes that can be exploited.

Remediation:

- Utilize global try-catch blocks to safely handle unexpected errors without revealing engine configurations.
- Provide clean, generic, customized error screens to end-users (e.g., "An internal error occurred").
- Keep technical debugging context, component paths, and system stack traces confined safely to backend logs.

Exploitation Scenario:

A database server goes offline briefly, causing a web application to fail and display a raw stack trace to the browser, exposing internal database connection parameters, usernames, and folder paths.

End of Document • Prepared by Cyber Security Engineering Core Team • Confidential and Proprietary